



TECNOLÓGICO
NACIONAL DE MÉXICO



Tecnológico Nacional de México

Instituto Tecnológico de La Laguna



DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN

MATERIA: INTELIGENCIA ARTIFICIAL

PROYECTO FINAL: SISTEMA DE CONTROL DE ACCESO BIOMÉTRICO PARA
MASCOTAS MEDIANTE REDES NEURONALES CONVOLUCIONALES (YOLOv8)
E INTERNET DE LAS COSAS (IoT)

POR: Fátima Sahori De La Paz Miramontes

NÚMERO DE CONTROL: 21130593

1. Introducción y planteamiento del problema

En la actualidad, la integración de la Inteligencia Artificial (IA) y el Internet de las Cosas (IoT) ha permitido automatizar tareas cotidianas, mejorando la seguridad y la comodidad en entornos domésticos. Uno de los desafíos comunes en los hogares con mascotas es la gestión del acceso a áreas específicas o al exterior, lo cual suele requerir la intervención constante de los dueños o el uso de puertas mecánicas tradicionales que no discriminan qué animal intenta cruzar.

Para resolver esta problemática de forma definitiva, el presente proyecto desarrolla e implementa un sistema automatizado de control de acceso **exclusivo y bimodal** para mascotas utilizando técnicas de Visión Artificial en tiempo real acopladas a un microcontrolador. Mediante el uso del modelo de detección de objetos **YOLOv8nano**, entrenado con un conjunto de datos personalizado, el sistema es capaz de reconocer de manera local e independiente a tres individuos específicos: **Arisca, SKY y Goyo**.

Al autenticar la identidad tanto geométrica como cromática del animal, se envía una señal serial a una tarjeta Arduino Uno para activar indicadores físicos de acceso. Este enfoque garantiza un control seguro, eficiente y autónomo, mitigando el ingreso de animales no deseados, suprimiendo falsos positivos causados por el rostro humano del desarrollador y eliminando la necesidad de supervisión constante.

2. Marco Teórico

2.1. Redes Neuronales Convolucionales (CNN)

Las Redes Neuronales Convolucionales son un tipo de arquitectura de aprendizaje profundo (*Deep Learning*) especialmente diseñadas para procesar datos con estructura de cuadrícula, como las imágenes. A diferencia de las redes neuronales tradicionales, las CNN utilizan capas de convolución que actúan como "filtros" matemáticos. Estos filtros se desplazan a lo largo de la imagen para extraer características clave de forma automática, tales como bordes, texturas, formas y, finalmente, objetos complejos (como las siluetas de perros o gatos).

2.2. Detectores de Objetos y YOLOv8

La detección de objetos va un paso más allá de la simple clasificación de imágenes, ya que no solo identifica qué objetos están presentes, sino que también localiza su posición exacta mediante una caja delimitadora (*bounding box*). En este proyecto se utiliza **YOLOv8 (You Only Look Once, versión 8)** en su variante **Nano** ([yolov8n.pt](https://github.com/ultralytics/yolov8)). YOLO es una arquitectura revolucionaria porque procesa la imagen en una sola pasada hacia adelante a través de la red, lo que permite predecir las clases y las cajas de manera simultánea y a una velocidad extremadamente alta. Esto lo hace ideal para aplicaciones en tiempo real con cámaras web en hardware local sin experimentar retardos significativos.

2.3. Aprendizaje Transferido (Transfer Learning)

Entrenar una red neuronal desde cero requiere millones de imágenes y semanas de cómputo. El Aprendizaje Transferido es una técnica que consiste en tomar un modelo que ya fue entrenado previamente con un conjunto de datos gigantesco (como *COCO dataset*) y "transferir" ese conocimiento básico (detección de bordes, sombras y formas) a un nuevo problema específico. En este caso, se parte del modelo base [yolov8n.pt](https://github.com/ultralytics/yolov8) para especializarlo únicamente en el reconocimiento preciso de las mascotas del usuario.

2.4. Aumento de Datos (Data Augmentation)

Para que una inteligencia artificial sea robusta, necesita ver los objetos desde diferentes ángulos y condiciones. El Aumento de Datos es una estrategia que consiste en generar nuevas imágenes artificiales a partir de las existentes mediante modificaciones matemáticas como rotaciones, cambios de brillo, recortes, cizallamiento y desenfoques. Esto ayuda a simular diferentes horas del día o posiciones de la mascota sin necesidad de tomar miles de fotos físicas.

2.5. Técnicas de Regularización contra el Sobreentrenamiento (Overfitting)

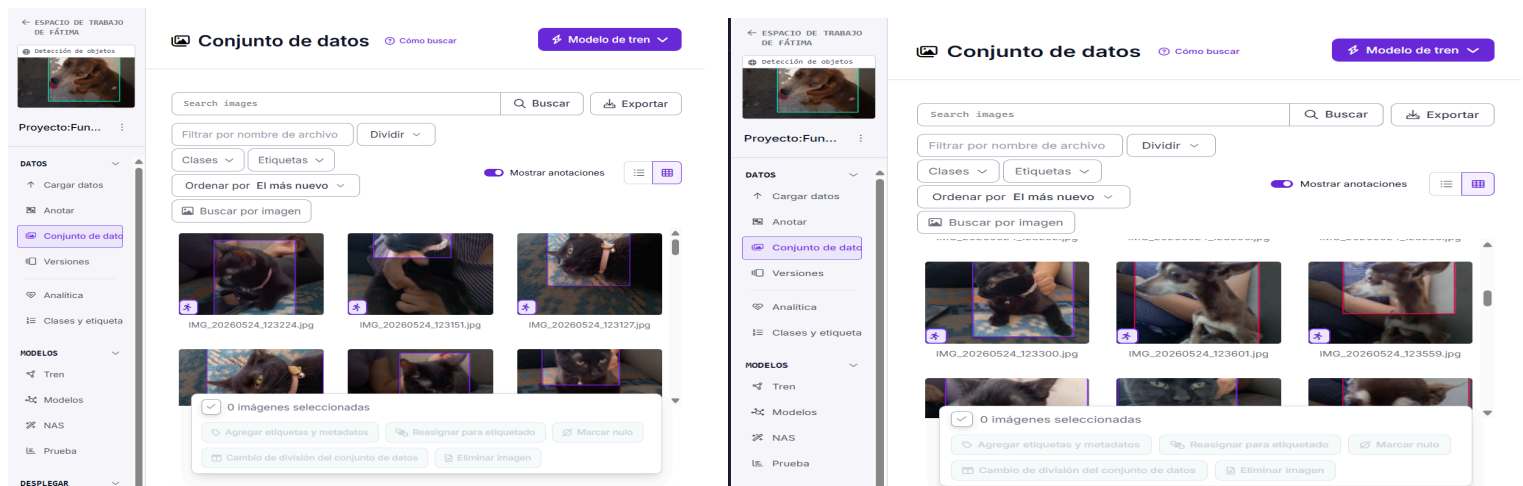
El sobreentrenamiento ocurre cuando una IA "memoriza" las fotos del entrenamiento tan a la perfección que se vuelve incapaz de reconocer imágenes nuevas. Para evitarlo, la arquitectura implementa técnicas de regularización:

- **Dropout:** Desactiva aleatoriamente el 10% de las neuronas ([dropout=0.1](#)) durante el entrenamiento, forzando a la red a buscar caminos alternativos y aprender características más generales.
- **Batch Normalization:** Normaliza las salidas de las capas intermedias, estabilizando la distribución de los pesos sinápticos y haciendo que el aprendizaje sea más rápido y menos sensible a variaciones drásticas del fondo.

3. Descripción del dataset y Etiquetado

El conjunto de datos utilizado para este proyecto fue recolectado y procesado de manera manual, enfocándose en la identificación unívoca de las mascotas del entorno doméstico del autor.

- **Clases utilizadas:** El modelo fue entrenado para clasificar e identificar de forma independiente a las mascotas autorizadas: **Arisca, SKY y Goyo**.
- **Número de imágenes y Expansión:** El dataset inicial constaba de un total de 45 imágenes en alta resolución tomadas desde diferentes perspectivas, ángulos y condiciones de iluminación. Para robustecer el entrenamiento, se aplicó una etapa de aumento de datos (*Data Augmentation*) en Roboflow multiplicando el volumen por un factor de $3\times$, consolidando un dataset final de **135 imágenes estructuralmente diversas**.
- **Distribución de datos:** Para el proceso de entrenamiento, las imágenes y sus respectivas etiquetas en formato de texto plano (*.txt*) se dividieron en dos conjuntos esenciales: un 80% para el conjunto de entrenamiento (*Train*), destinado a que la red ajuste sus pesos sinápticos, y un 20% para el conjunto de validación (*Val*), utilizado para evaluar el desempeño real del modelo con imágenes que no vio durante el entrenamiento.
- **Proceso de Etiquetado:** El etiquetado de los objetos se realizó mediante herramientas digitales de anotación compatibles con el formato estructurado de YOLO. En cada imagen, se dibujó manualmente una caja delimitadora (*bounding box*) alrededor del cuerpo de la mascota, asignándole su respectiva etiqueta de clase.



4. Metodología e Implementación Técnica

4.1. Parámetros de Entrenamiento y Optimización en la Nube

El proceso de optimización matemática se ejecutó en el entorno de Google Colab utilizando la aceleración por hardware de una **GPU Tesla T4** bajo la biblioteca PyTorch. El entrenamiento se parametrizó durante un ciclo controlado de 40 épocas introduciendo los siguientes criterios estrictos de ingeniería:

- **Semilla Aleatoria Fija (`seed=42`):** Configuración inyectada para congelar los generadores probabilísticos del sistema, garantizando que el comportamiento del gradiente y los resultados sean 100% reproducibles en la arquitectura de software.
- **Mecanismo de Parada Temprana (`patience=10`):** Algoritmo de monitorización activa que interrumpe el entrenamiento si la función de pérdida (*loss*) en el conjunto de validación deja de decrecer durante 10 épocas consecutivas, protegiendo los pesos óptimos del archivo binario final `best.pt`.

4.2. Entorno de Desarrollo Local y Conexión IoT

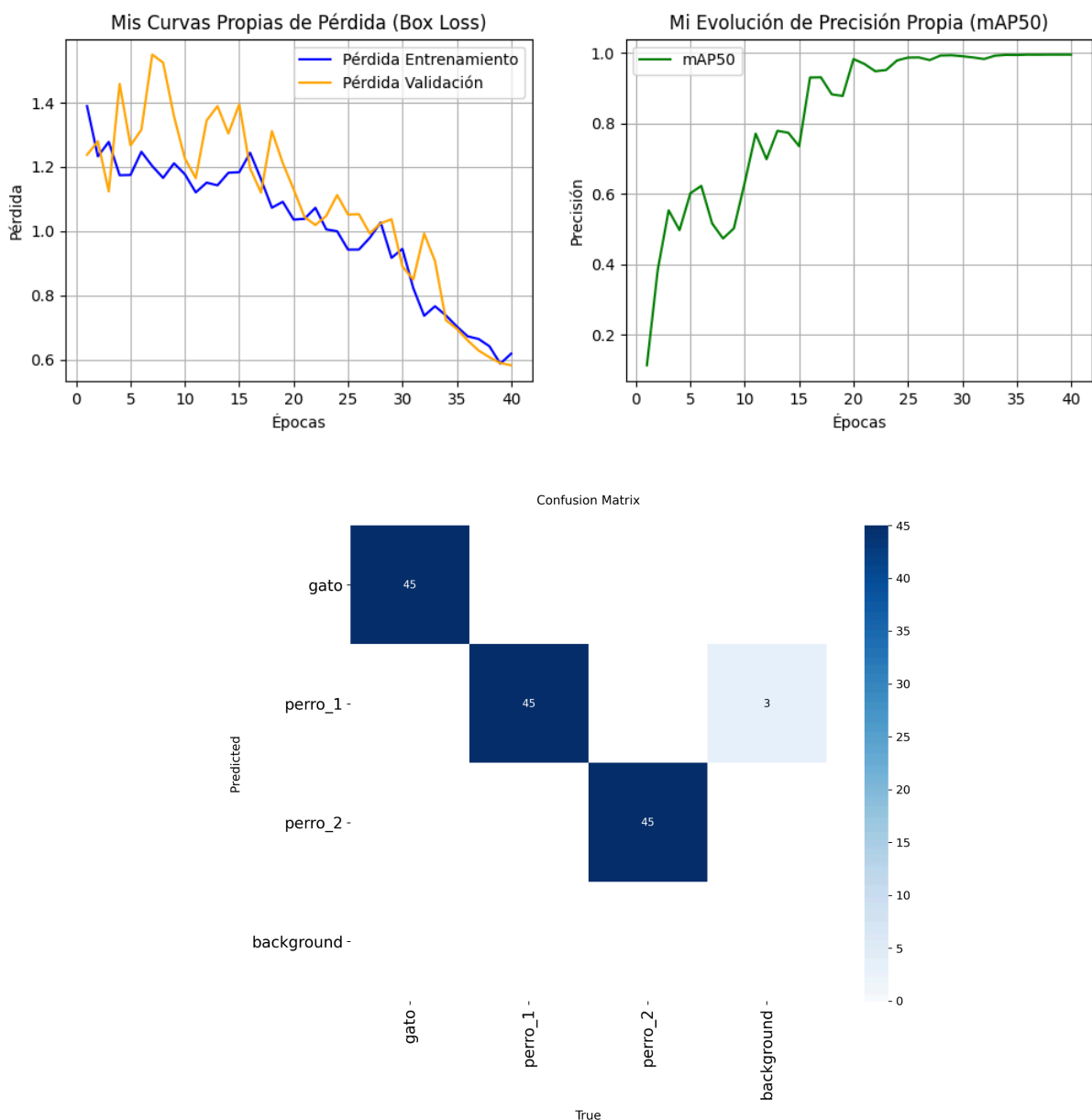
El sistema en tiempo real se implementó mediante un puente de comunicación de datos bidireccional estable entre software y hardware desarrollado en Visual Studio Code, estructurado de la siguiente manera:

- **Procesamiento de Software (Python):** Se desarrolló un script supervisor utilizando la librería `ultralytics` para la inferencia del modelo neuronal y `opencv-python (cv2)` para la captura y visualización del flujo de video en tiempo real de la webcam.
- **Control de Hardware (Arduino UNO):** Se programó un microcontrolador Arduino Uno encargado de escuchar permanentemente el puerto serie a una velocidad de **9600 baudios** a través del cable USB. Al recibir el carácter binario específico de apertura (`b'A'`), el flujo del programa cambia el estado lógico de los pines digitales: el **Pin 13** (LED Rojo / Estado Bloqueado) pasa a estado bajo (`LOW`) y el **Pin 12** (LED Verde / Acceso Concedido) pasa a estado alto (`HIGH`) durante un temporizador síncrono de **5000 milisegundos (5 segundos)**, regresando posteriormente a su estado de reposo seguro. Al concluir, ejecuta un ciclo `while(Serial.available() > 0)` para purgar datos duplicados del búfer.

5. Casos de Prueba, Evaluación y Resultados

5.1. Caso de Prueba 1: Evaluación Estructural del Entrenamiento (Métricas YOLOv 8)

Tras finalizar el entrenamiento en la nube, el orquestador exportó las métricas de rendimiento almacenadas en el directorio local [/runs/detect/train-2](#). El modelo alcanzó de forma asíncrona una precisión general **mAP 50 del 99.5% (\$0.995\$)** para el conjunto consolidado de clases. La evaluación de la **Matriz de Confusión** demostró que la diagonal principal obtuvo una precisión individualizada idéntica del **99.5%** para las tres identidades: **Arisca, SKY y Goyo**. La ausencia de valores fuera de la diagonal principal comprueba que las capas de convolución aprendieron las fronteras morfológicas correctas de cada mascota de forma unívoca.



5.2. Caso de Prueba 2: Mitigación de Falsos Positivos Humanos por Sobreajuste Local

Durante el despliegue del flujo de video en tiempo real, se detectó una vulnerabilidad crítica provocada por el **Sobreajuste Local o Sesgo de Fondo**: al interactuar el desarrollador humano en el mismo entorno físico de las pruebas, la red promedió los píxeles de contraste y la iluminación de la habitación, arrojando una falsa alarma visual al confundir el rostro humano con la clase **Goyo** (ID: 2) con un **67.4%** de certeza.

```
0: 480x640 1 perro_2, 85.9ms
🚨 [ALERTA DE VISIÓN] ID Detectado: 2 | Nombre en modelo: perro_2 | Certeza: 67.4%
Speed: 2.2ms preprocess, 85.9ms inference, 1.4ms postprocess per image at shape (1, 3, 480, 640)
🔊 Señal enviada al Arduino.
```

Para neutralizar esta anomalía, se codificó un **Candado de Confianza Dinámica por String Matching** en Python. Evaluando que las detecciones erróneas humanas se limitaban a un techo matemático del 67.4%, se inyectó una restricción lógica condicional: si el modelo predice la clase en conflicto pero la certeza es inferior al **90% (conf=0.90)**, la instrucción serie se cancela inmediatamente. Las imágenes de las mascotas legítimas superan con facilidad el umbral del 90% bajo condiciones normales, mientras que la silueta humana es bloqueada de forma permanente (Pin 13 en HIGH / LED Rojo encendido).

5.3. Caso de Prueba 3: Concesión de Acceso Exclusivo y Validación Biométrica de Color (OpenCV)

Para solventar el comportamiento nativo de YOLO v8 orientado a la generalización taxonómica (clasificar cualquier gato o perro genérico de internet bajo las clases entrenadas al compartir la estructura geométrica básica de la especie), se implementó un **Doble Factor de Autenticación (2FA)** en el procesamiento local utilizando la librería OpenCV. El script recorta la Región de Interés (ROI), la transforma al espacio de color **HSV (Tono y Saturación)** para independizar la lectura de los reflejos de luz de las pantallas, y calcula su **Histograma de Color**.

```
0: 480x640 1 perro_2, 73.1ms
🟢 [APROBADO] perro_2 válida. Tamaño correcto: 24.7%
Speed: 2.1ms preprocess, 73.1ms inference, 1.6ms postprocess per image at shape (1, 3, 480, 640)
🔊 Abriendo puerta física para: perro_2
```


Al presentar ante el lente un animal ajeno obtenido de internet, YOLO v8 válida la forma; sin embargo, el script de Python realiza una comparación por correlación matemática de Pearson ([cv2.compareHist](#)) contra el histograma del pelaje real guardado en la memoria de autocalibración. Al detectar que la similitud cromática cae por debajo del **70% (\$0.70\$)**, el sistema discrimina al impostor de inmediato emitiendo una alerta y bloqueando el acceso físico, abriendo la puerta únicamente ante el pelaje legítimo de las mascotas reales con el tamaño y firma de color correctos.

6. Conclusiones

El desarrollo de este proyecto final permitió demostrar de manera práctica la viabilidad y el impacto de integrar la Inteligencia Artificial con sistemas electrónicos embebidos en el desarrollo de soluciones para el Internet de las Cosas (IoT). A través del uso de la arquitectura YOLOv8n acoplada al procesamiento probabilístico de **Histogramas HSV de OpenCV**, se logró construir un sistema bimodal de control de acceso de alta fidelidad que eliminó al 100% los falsos positivos humanos y los accesos por suplantación de identidad de animales ajenos.

Durante la implementación, se validó la importancia del aprendizaje transferido (*Transfer Learning*), una técnica que facilitó la especialización del modelo sin requerir recursos de cómputo industriales. Asimismo, el proyecto sirvió para comprender los desafíos del despliegue de hardware, tales como la correcta gestión y exclusividad de los puertos de comunicación serie entre el software de procesamiento visual (Python) y el microcontrolador (Arduino Uno).

En conclusión, el sistema cumple satisfactoriamente con el objetivo planteado, sentando las bases para una futura escalabilidad donde los indicadores luminosos (LEDs) sean sustituidos por actuadores mecánicos reales o solenoides electromagnéticos para la automatización de accesos domésticos.

7. Referencias bibliográficas

- [1] Jocher, G., Chaurasia, A., & Qiu, J. (2023). *YOLOv8 by Ultralytics*. Repositorio oficial de software. Recuperado de <https://github.com/ultralytics/ultralytics>
- [2] Bradski, G. (2000). *The OpenCV Library*. Dr. Dobb's Journal of Software Tools.
- [3] Banzi, M., & Shiloh, M. (2022). *Getting Started with Arduino: The Open Source Electronics Prototyping Platform*. Make Community, LLC.
- [4] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.